

Conceitos básicos sobre Maven, GitHub, GitHub Actions

SUMÁRIO

Seção 1 – Maven	3
Seção 2 – GitHub	7
Seção 3 – GitHub Actions	13
Exercício	16

SEÇÃO 1 – MAVEN

Apache Maven é uma ferramenta de gerenciamento e construção de projetos de software. Com base no conceito de modelo de objeto de projeto (POM), o Maven pode gerenciar a construção, os relatórios e a documentação de um projeto a partir de uma informação central.

O Maven surgiu da necessidade de criar uma maneira padrão de construir os projetos, uma definição clara do que consistia o projeto, uma maneira fácil de publicar informações do projeto e uma maneira de compartilhar JARs entre vários projetos.

O resultado é uma ferramenta que agora pode ser usada para construir e gerenciar qualquer projeto baseado em Java. Portanto o Maven apoia os desenvolvedores no processo de construção, compilação, empacotamento, testes e distribuição de seus projetos.

Objetivos:

O objetivo principal do Maven é permitir que um desenvolvedor compreenda o estado completo de um esforço de desenvolvimento no menor período de tempo. Para atingir esse objetivo, o Maven lida com diversas áreas de preocupação:

- Facilitando o processo de construção
- Fornecendo um sistema de construção uniforme
- Fornecendo informações de projeto de qualidade
- Incentivando melhores práticas de desenvolvimento
- Facilitando o processo de construção

Embora o uso do Maven não elimine a necessidade de conhecer os mecanismos subjacentes, o Maven protege os desenvolvedores de muitos detalhes.

Fornecendo um sistema de construção uniforme

Maven constrói um projeto usando seu modelo de objeto de projeto (POM) e um conjunto de plugins. Depois de se familiarizar com um projeto Maven, você saberá como todos os projetos Maven são construídos. Isso economiza tempo ao navegar em muitos projetos.

Fornecendo informações de projeto de qualidade

O Maven fornece informações úteis sobre o projeto que são parcialmente retiradas do seu POM e parcialmente geradas a partir das fontes do seu projeto. Por exemplo, Maven pode fornecer:

- Log de alterações criado diretamente do controle de origem
- Fontes de referência cruzada
- Listas de discussão gerenciadas pelo projeto
- Dependências usadas pelo projeto
- Relatórios de testes unitários incluindo cobertura

Produtos de análise de código de terceiros também fornecem plug-ins Maven que adicionam seus relatórios às informações padrão fornecidas pelo Maven.

Fornecendo diretrizes para o desenvolvimento de melhores práticas

O Maven visa reunir os princípios atuais para o desenvolvimento de melhores práticas e facilitar a orientação de um projeto nessa direção.

Por exemplo, ***especificação, execução e relatório de testes unitários*** fazem parte do ciclo normal de construção usando Maven. As melhores práticas atuais de testes unitários foram usadas como diretrizes:

- Manter o código-fonte de teste em uma árvore de origem separada, mas paralela
- Usando convenções de nomenclatura de casos de teste para localizar e executar testes
- Fazer com que os casos de teste configurem seu ambiente em vez de personalizar a construção para preparação do teste

Resumo dos recursos

A seguir estão os principais recursos do Maven em poucas palavras:

- Configuração simples do projeto que segue as práticas recomendadas - inicie um novo projeto ou módulo em segundos
- Uso consistente em todos os projetos - significa que não há tempo de aceleração para novos desenvolvedores entrarem em um projeto
- Gerenciamento superior de dependências, incluindo atualização automática e fechamento de dependências (também conhecidas como dependências transitivas)
- Capaz de trabalhar facilmente com vários projetos ao mesmo tempo
- Um grande e crescente repositório de bibliotecas e metadados para uso imediato, e acordos em vigor com os maiores projetos de código aberto para disponibilidade em tempo real de seus lançamentos mais recentes

- Extensível, com a capacidade de escrever plug-ins facilmente em Java ou linguagens de script
- Acesso instantâneo a novos recursos com pouca ou nenhuma configuração extra
- Tarefas Ant para gerenciamento de dependências e implantação fora do Maven
- Construções baseadas em modelo: o Maven é capaz de construir qualquer número de projetos em tipos de saída predefinidos, como JAR, WAR ou distribuição com base em metadados sobre o projeto, sem a necessidade de fazer qualquer script na maioria dos casos.
- Site coerente de informações do projeto: Usando os mesmos metadados do processo de construção, o Maven é capaz de gerar um site ou PDF incluindo qualquer documentação que você queira adicionar e adiciona a isso relatórios padrão sobre o estado de desenvolvimento do projeto. Exemplos destas informações podem ser vistos na parte inferior da navegação esquerda deste site, nos submenus “Informações do Projeto” e “Relatórios do Projeto”.
- Gerenciamento de lançamento e publicação de distribuição: Sem muita configuração adicional, o Maven se integrará ao seu sistema de controle de origem (como Subversion ou Git) e gerenciará o lançamento de um projeto com base em uma determinada tag. Ele também pode publicar isso em um local de distribuição para uso por outros projetos. O Maven é capaz de publicar saídas individuais, como um JAR, um arquivo incluindo outras dependências e documentação, ou como uma distribuição fonte.
- Gerenciamento de dependências: Maven incentiva o uso de um repositório central de JARs e outras dependências. O Maven vem com um mecanismo que os clientes do seu projeto podem usar para baixar quaisquer JARs necessários para construir seu projeto a partir de um repositório JAR central, muito parecido com o CPAN do Perl. Isso permite que os usuários do Maven reutilizem JARs entre projetos e incentiva a comunicação entre projetos para garantir que os problemas de compatibilidade com versões anteriores sejam resolvidos.

Ciclo de vida de construção

O Maven é baseado no conceito central de ciclo de vida de construção. O que isto significa é que o processo de construção e distribuição de um determinado artefato (projeto) está claramente definido.

Para quem está construindo um projeto, isso significa que é necessário apenas aprender um pequeno conjunto de comandos para construir qualquer projeto Maven, e o POM garantirá que eles obtenham os resultados desejados.

Existem três ciclos de vida de construção integrados: *default*, *clean* e *site*. O ciclo de vida *default* cuida da implantação do seu projeto, o ciclo de vida *clean* cuida da limpeza do projeto, enquanto o ciclo de vida *site* cuida da criação do site do seu projeto.

Fases do ciclo de vida de construção

Cada um desses ciclos de vida de construção é definido por uma lista diferente de fases de construção, em que uma fase de construção representa um estágio no ciclo de vida.

Por exemplo, o ciclo de vida *default* compreende as seguintes fases:

- **validate** - validar se o projeto está correto e se todas as informações necessárias estão disponíveis
- **compile** - compilar o código fonte do projeto
- **test** - testar o código-fonte compilado usando uma estrutura de teste de unidade adequada. Esses testes não devem exigir que o código seja empacotado ou implantado
- **package** - pegue o código compilado e empacote-o em seu formato distribuível, como um JAR.
- **verify** - executar quaisquer verificações nos resultados dos testes de integração para garantir que os critérios de qualidade sejam atendidos
- **install** - instale o pacote no repositório local, para uso como dependência em outros projetos localmente
- **deploy** - feito no ambiente de construção, copia o pacote final para o repositório remoto para compartilhar com outros desenvolvedores e projetos.

Essas fases do ciclo de vida (mais as outras fases do ciclo de vida não mostradas aqui) são executadas sequencialmente para completar o ciclo de vida *default*. Dadas as fases do ciclo de vida acima, isso significa que quando o ciclo de vida *default* é usado, o Maven primeiro validará o projeto, depois tentará compilar os fontes, executá-los nos testes, empacotar os binários (por exemplo, jar), executar testes de integração nesse pacote, verificar os testes de integração, instale o pacote verificado no repositório local e, em seguida, implante o pacote instalado em um repositório remoto.

Plugins

As funcionalidades do Maven podem ser estendidas através de componentes modulares chamados *plugins*. Estes *plugins* podem ser inseridos no projeto para que execute uma tarefa específica dentro do ciclo de vida do projeto. Os *plugins* são configurados no arquivo POM (Project Object Model), que descreve o projeto e suas dependências, e são baixados automaticamente do repositório central do *Maven* quando necessário. Com os *plugins*, o *Maven* torna-se altamente adaptável e flexível para atender a diferentes necessidades de construção e gerenciamento de projetos.

Fonte: <https://maven.apache.org/>

[CLIQUE AQUI E VEJA A VIDEO-AULA COM UMA DEMONSTRAÇÃO DE USO](#)

SEÇÃO 2 – GitHub

O que é o GitHub?

O GitHub é uma plataforma online que suporta um conjunto de serviços baseado em nuvem, e uma das suas principais funções, é o fornecer o serviço de Sistema de Controle de Versão (VSC) chamado Git. Com o GitHub é possível que desenvolvedores colaborem e desenvolvam projetos em conjunto, tendo um alto detalhamento e histórico de todas as alterações e progresso de seus projetos. É amplamente utilizado por desenvolvedores e equipes de desenvolvimento como repositório para integrar códigos-fonte, mas também é utilizado para colaborar, gerenciar e acompanhar o progresso de projetos de software.

Sistema de Controle de Versões - Git

Git é um sistema de controle de versão distribuído que rastreia alterações em qualquer conjunto de arquivos de computador, geralmente usado para coordenar o trabalho entre desenvolvedores que criam código-fonte de forma colaborativa durante o desenvolvimento de software. Seus objetivos incluem velocidade, integridade de dados e suporte para fluxos de trabalho distribuídos e não lineares. O Git fornece uma interface de linha de comando (CLI) que pode ser utilizado tanto no Windows, Linux ou iOS.

Serviços do GitHub:

O Github oferece diversos serviços para Colaboração de Código, Automação e Integração Contínua/Entrega Contínua (CI/CD), Segurança, Aplicativos, Gerenciamento de Projetos e Administração de Times, abaixo está uma lista com um breve descritivo de cada serviço:

Colaboração de Código:

- **Codespaces:** Crie ambientes de desenvolvimento totalmente configurados na nuvem com todo o poder do seu editor favorito.
- **GitHub Copilot:** obtenha sugestões para linhas inteiras ou funções inteiras diretamente no seu editor.
- **Pull requests:** Permita que os contribuidores/desenvolvedores notifiquem você facilmente sobre alterações que eles enviaram para um repositório – com acesso limitado aos usuários que você especificar. Mescle facilmente as alterações que você aceita.

- Discussions: Espaço dedicado para sua comunidade se reunir, fazer e responder perguntas e ter conversas abertas.
- Code search: permite que os desenvolvedores pesquisem, naveguem e entendam rapidamente o código diretamente pelo site do GitHub.
- Notifications: Receba atualizações sobre as atividades do GitHub que você assinou. Use a caixa de entrada de notificações para personalizar, fazer a triagem e gerenciar suas atualizações.
- Code review: Revise o novo código, veja alterações visuais no código e mescle alterações de código com segurança com verificações de status automatizadas.
- Code review assignments: Atribua revisões de código para deixar claro quais membros da equipe devem enviar sua revisão para um pull request.
- Code owners: Solicite revisões automaticamente – ou exija aprovação – de colaboradores selecionados quando alterações forem feitas em seções de código de sua propriedade.
- Draft pull requests: Use uma solicitação pull como uma forma de discutir e colaborar, sem submeter-se a uma revisão formal ou arriscar uma mesclagem indesejada.
- Protected branches: Imponha restrições sobre como os ramos de código são mesclados, incluindo a exigência de revisões ou a permissão de que apenas colaboradores específicos trabalhem em um ramo específico.
- Team discussions: Publique e discuta atualizações em toda a sua organização GitHub ou apenas em sua equipe. Notifique os participantes com atualizações e conecte-se de qualquer lugar.
- Team reviewers: Solicite uma equipe no GitHub para revisar sua solicitação pull. Os membros da equipe receberão uma notificação indicando que você solicitou a revisão.
- Multiple assignees: Atribua até 10 pessoas para trabalhar em um determinado problema ou pull request, permitindo rastrear mais facilmente quem está trabalhando em quê.
- Multiple reviewers: Solicite revisão de vários colaboradores. Os revisores solicitados serão notificados de que você solicitou a revisão.
- Multi-line comments: Esclareça as revisões de código fazendo referência ou comentando em várias linhas de uma vez em uma visualização de comparação de pull request.
- Public repositories: Trabalhe com qualquer membro do GitHub no código de um repositório público que você controla. Faça alterações, abra uma solicitação pull, crie um problema e muito mais.
- Dark mode: Mude para o tema escuro ou use como padrão as preferências do sistema.

Automação e Integração Contínua/Entrega Contínua (CI/CD)

- **Actions:** Automatize todos os seus fluxos de trabalho de desenvolvimento de software. Escreva tarefas e combine-as para criar, testar e implantar com mais rapidez no GitHub.
- **Packages:** Hospede seus próprios pacotes de software ou use-os como dependências em outros projetos. Hospedagem privada e pública disponível.
- **APIs:** Crie chamadas para obter todos os dados e eventos necessários no GitHub e inicie e avance automaticamente seus fluxos de trabalho de software.
- **GitHub Pages:** Crie e publique sites sobre você, sua organização ou seu projeto diretamente de um repositório GitHub.
- **GitHub Marketplace:** Comece com milhares de actions e aplicativos da nossa comunidade para ajudá-lo a criar, melhorar e acelerar seus fluxos de trabalho automatizados.
- **Webhooks:** Dezenas de eventos e uma API de Webhooks ajudam você a integrar e automatizar o trabalho de seu repositório, organização ou aplicativo.
- **Hosted runners:** Mova a automação para a nuvem com ambientes Linux, Windows e MacOS sob demanda para suas execuções de fluxo de trabalho, hospedados pelo GitHub.
- **Self-hosted runners:** Mais ambientes e controle mais completo com rótulos, grupos e políticas para gerenciar execuções em suas próprias máquinas. Além disso, o aplicativo runner é de código aberto.
- **Secrets management:** Compartilhe, atualize e sincronize segredos automaticamente em vários repositórios para aumentar a segurança e reduzir falhas no fluxo de trabalho.
- **Environments:** Atenda aos requisitos de segurança e conformidade para entrega com segredos e regras de proteção.
- **Deployments:** Veja qual versão do seu código está em execução em um ambiente, incluindo quando e por quê, além de logs para revisão.
- **Workflow visualization:** Mapeie fluxos de trabalho, acompanhe sua progressão em tempo real, entenda fluxos de trabalho complexos e comunique o status com o restante da equipe.
- **Workflow templates:** Padronize e dimensione práticas e processos recomendados com modelos de fluxo de trabalho pré-configurados compartilhados por toda a sua organização.
- **Policies:** Gerencie o uso e as permissões de ações por repositório e organizações, com políticas adicionais para pull requests de fork.

Segurança:

- Private repos - Hospede o código que você não deseja compartilhar com o mundo em repositórios privados do GitHub, acessíveis apenas para você e para as pessoas com quem você os compartilha.
- 2FA: Adicione uma camada extra de segurança com autenticação de dois fatores (2FA) ao fazer login no GitHub. Exija 2FA e escolha entre aplicativos TOTP, chaves de segurança e muito mais.
- Required reviews: Garanta que as solicitações pull tenham um número específico de revisões de aprovação antes que os colaboradores possam fazer alterações em uma ramificação protegida.
- Required status checks: Certifique-se de que todos os testes de CI necessários sejam aprovados antes que os colaboradores possam fazer alterações em uma ramificação protegida.
- Code scanning: Encontre vulnerabilidades em código personalizado usando análise estática. Evite a introdução de novas vulnerabilidades verificando cada solicitação pull.
- Secret scanning: Encontre segredos codificados em seus repositórios públicos e privados. Revogue-os para manter seguro o acesso aos serviços que você usa.
- Private vulnerability reporting: Permita que seu repositório público receba relatórios de vulnerabilidade da comunidade de forma privada e colabore em uma solução.
- Dependency graph: See the packages your project depends on, the repositories that depend on them, and any vulnerabilities detected in their dependencies.
- Dependabot Alerts: Seja notificado quando houver novas vulnerabilidades afetando seus repositórios. O GitHub detecta e alerta os usuários sobre dependências vulneráveis em repositórios públicos e privados.
- Dependabot security and version updates: Mantenha sua cadeia de suprimentos segura e atualizada abrindo automaticamente solicitações pull que atualizam dependências vulneráveis ou desatualizadas.
- Dependency review: Entenda o impacto na segurança de dependências recém-introduzidas durante solicitações pull, antes que elas sejam mescladas.
- GitHub Security Advisories: Relate, discuta, corrija e publique informações sobre vulnerabilidades de segurança encontradas em repositórios de código aberto de maneira privada.
- GitHub Advisory Database: Navegue ou pesquise as vulnerabilidades que o GitHub conhece. O banco de dados contém todos os CVEs selecionados e avisos de segurança no gráfico de dependência do GitHub.
- GPG commit signing verification: Use GPG ou S/MIME para assinar tags e commits localmente. Eles são marcados como verificados no GitHub para que outras pessoas saibam que as alterações vêm de uma fonte confiável.

- Security audit log: Revise rapidamente as ações executadas pelos membros da sua organização. Seu registro de auditoria inclui detalhes como quem executou uma ação e quando.

Aplicativos:

- GitHub Mobile - Leve seus projetos, ideias e códigos para uma experiência totalmente nativa em dispositivos móveis e tablets.
- CLI do GitHub – leve o GitHub para a linha de comando. Gerencie problemas e receba solicitações do terminal, onde você já está trabalhando com Git e seu código.
- GitHub Desktop – Simplifique seu fluxo de trabalho de desenvolvimento com uma GUI. Visualize, confirme e envie alterações sem nunca tocar na linha de comando.

Gerenciamento de projetos:

- Projects: Crie uma visão personalizada de seus problemas e receba solicitações para planejar e acompanhar seu trabalho.
- Labels: Organize e priorize seu trabalho. Aplique rótulos a problemas e receba solicitações para indicar prioridade, categoria ou qualquer outra informação que você achar útil.
- Milestones: Acompanhe o progresso em grupos de problemas ou pull requests em um repositório e mapeie grupos para metas gerais do projeto.
- Issues: Rastreie bugs, melhorias e outras solicitações, priorize o trabalho e comunique-se com as partes interessadas à medida que as alterações são propostas e mescladas.
- Charts and Insights: Aproveite os insights para visualizar seus projetos criando e compartilhando gráficos criados a partir dos dados do seu projeto.
- Tasklists: decomponha problemas em tarefas, converta tarefas em problemas, visualize seus relacionamentos em projetos GitHub e muito mais, tudo em uma nova interface de usuário sofisticada.
- Org dependency insights: Com insights de dependência, você pode visualizar vulnerabilidades, licenças e outras informações importantes para os projetos de código aberto dos quais sua organização depende.
- Repo insights: Use dados sobre atividades e contribuições em seus repositórios, incluindo tendências, para fazer melhorias baseadas em dados em seu ciclo de desenvolvimento.
- Wikis: Hospede documentação para projetos em um wiki dentro do seu repositório. Os colaboradores podem editar facilmente a documentação na web ou localmente.

Administração de times:

- Organizations: Configure grupos de contas de usuários que possuem repositórios. Gerencie o acesso equipe por equipe ou usuário individual.
- Invitations: Adicione facilmente membros do GitHub aos seus repositórios usando seu nome de usuário ou endereço de e-mail do GitHub e solicite que eles confirmem o acesso.
- Teams: Agrupe os membros da sua organização para refletir a estrutura da sua empresa ou grupo com permissões e menções de acesso em cascata.
- Team sync: Habilite a sincronização de equipe entre seu provedor de identidade e sua organização no GitHub, incluindo Azure AD e Okta.
- Custom roles: Defina o nível de acesso dos usuários ao seu código, dados e configurações com base na função deles na sua organização.
- Custom repository roles: Garanta que os membros tenham apenas as permissões necessárias criando funções personalizadas com configurações de permissão refinadas.
- Domain verification: Verifique a identidade da sua organização no GitHub e exiba essa verificação por meio de um crachá de perfil.
- Verified and approved domains: Certifique-se de que os e-mails cheguem apenas à caixa de entrada de e-mail da sua empresa, aprovando domínios corporativos.
- Audit Log API: revise rapidamente as ações realizadas pelos membros da sua organização. Monitore o acesso, alterações de permissão, alterações de usuário e outros eventos.
- Audit log streaming: Evite a perda de log de auditoria transmitindo seu log de auditoria corporativo com informações de sistema líderes, ferramentas de gerenciamento de eventos e provedores de armazenamento em nuvem.
- Repo creation restriction: restrinja as permissões de criação de repositório apenas aos proprietários da organização ou permita que os membros criem repositórios públicos e privados.
- Notification restriction: Proteja as informações sobre o que sua equipe está trabalhando, restringindo as notificações por e-mail a domínios de e-mail aprovados.
- Enterprise Accounts: Habilite a colaboração entre sua organização e os ambientes GitHub com um único ponto de visibilidade e gerenciamento por meio de uma conta corporativa.
- Compliance Reports: Cuide de suas necessidades de avaliação de segurança e certificação acessando os relatórios de conformidade de nuvem do GitHub,

como nossos relatórios SOC e autoavaliações CAIQ da Cloud Security Alliance (CSA CAIQ).

Fonte: <https://github.com>

[CLIQUE AQUI E VEJA A VIDEO-AULA COM UMA DEMONSTRAÇÃO DE USO](#)

SEÇÃO 3 – GitHub ACTIONS

GitHub Actions é uma plataforma de integração contínua e entrega contínua (CI/CD) que permite automatizar a sua compilação, testar e pipeline de implantação. É possível criar fluxos de trabalho que criam e testam cada pull request no seu repositório, ou implantar pull requests mesclados em produção.

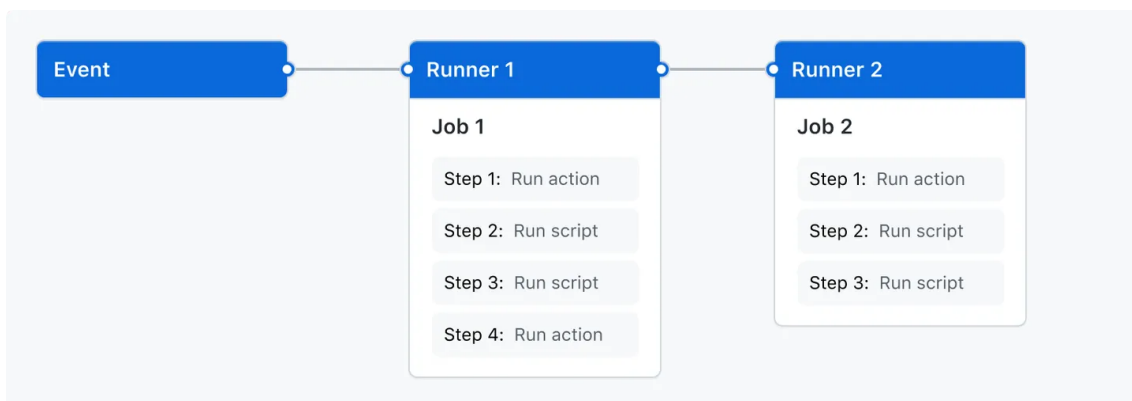
GitHub Actions vai além de apenas DevOps e permite que você execute fluxos de trabalho quando outros eventos ocorrerem no seu repositório. Por exemplo, você pode executar um fluxo de trabalho para adicionar automaticamente as etiquetas apropriadas sempre que alguém cria um novo problema no repositório.

GitHub fornece máquinas virtuais do Linux, Windows e macOS para executar seus fluxos de trabalho, ou você pode hospedar seus próprios executores auto-hospedados na sua própria infraestrutura de dados ou na nuvem.

Componentes de GitHub Actions

Você pode configurar um fluxo de trabalho do GitHub Actions a ser disparado quando um evento ocorrer no seu repositório, como a abertura de uma solicitação de pull ou a criação de uma Issue. Seu fluxo de trabalho contém um ou mais *Jobs* que podem ser executados em ordem sequencial ou em paralelo. Cada *job* será executado em um *runner* próprio de máquina virtual ou em um contêiner e tem um ou mais *steps* que executam um script definido por você ou uma ação, que é uma extensão reutilizável que pode simplificar o fluxo de trabalho.

Diagrama de um evento que dispara o *Runner 1* para executar o *Job 1*, que dispara o *Runner 2* para executar o *Job 2*. Cada um dos trabalhos é dividido em vários *steps*.



Fluxos de trabalho

Um fluxo de trabalho é um processo automatizado configurável que executa um ou mais *Jobs*. Os fluxos de trabalho são definidos por um arquivo YAML verificado no seu repositório e será executado quando acionado por um evento no repositório, ou eles podem ser acionados manualmente ou de acordo com um cronograma definido.

Os fluxos de trabalho são definidos no diretório `.github/workflows` em um repositório. Um repositório pode ter vários fluxos de trabalho, cada um dos quais pode executar um conjunto diferente de tarefas. Por exemplo, você pode ter um fluxo de trabalho para criar e testar pull requests, outro fluxo de trabalho para implantar seu aplicativo toda vez que uma versão for criada, e outro fluxo de trabalho que adiciona uma etiqueta toda vez que alguém abre um novo *Issue*.

Eventos

Um evento é uma atividade específica em um repositório que aciona a execução de um fluxo de trabalho. Por exemplo, a atividade pode originar-se de GitHub quando alguém cria uma solicitação de pull request, abre um *Issue* ou faz envio por push de um commit para um repositório. Você também pode disparar uma execução de fluxo de trabalho de acordo com um agendamento, com um POST em uma API REST ou manualmente.

Jobs

Um *job* é um conjunto de *steps* em um fluxo de trabalho executado no mesmo *runner*. Cada *step* é um script de shell que será executado ou uma *action* que será executada. Os *steps* são executados em ordem e dependem um do outro. Uma vez que cada *step* é executado no mesmo *runner*, você pode compartilhar dados de um *step* para outro. Por exemplo, você pode ter uma *step* que compila a sua aplicação seguida de um *step* que testa ao aplicativo criado.

Você pode configurar as dependências de um *job* com outros *jobs*; por padrão, os *jobs* não têm dependências e são executados em paralelo um com o outro. Quando um *job* leva uma dependência de outro *job*, ele irá aguardar que o *job* dependente seja concluído antes que possa executar. Por exemplo, você pode ter vários *jobs* de criação para diferentes arquiteturas que não têm dependências, e um *job* de pacotes que depende desses *jobs*. Os *jobs* de criação serão executados em paralelo e, quando todos forem concluídos com sucesso, o *job* de empacotamento será executado.

Actions

Uma *action* é um aplicativo personalizado para a plataforma GitHub Actions que executa uma tarefa complexa, mas frequentemente repetida. Use uma *action* para ajudar a reduzir a quantidade de código repetitivo que você grava nos seus arquivos de fluxo de trabalho. Uma ação pode extrair o seu repositório git de GitHub, configurar a cadeia de ferramentas correta para seu ambiente de criação ou configurar a autenticação para seu provedor de nuvem.

Você pode gravar suas próprias *actions*, ou você pode encontrar *actions* para usar nos seus fluxos de trabalho em GitHub Marketplace.

Runners

Um *runner* é um servidor que executa seus fluxos de trabalho quando são acionados. Cada *runner* pode executar uma tarefa por vez. GitHub fornece *runners* para Ubuntu Linux, Microsoft Windows e macOS para executar seus fluxos de trabalho. Cada fluxo de trabalho é executado em uma nova máquina virtual provisionada. GitHub também oferece runners maiores, que estão disponíveis em configurações maiores. Caso precise ter outro sistema operacional ou uma configuração de hardware específica, hospede seus próprios *runners*.

Na próxima aula, iremos criar um fluxo de trabalho que será disparado após um commit no repositório, este fluxo de trabalho irá efetuar a construção do projeto com o Maven.

Fonte: <https://github.com/features/actions>

[CLIQUE AQUI E VEJA A VIDEO-AULA COM UMA DEMONSTRAÇÃO DE USO](#)